**Rapport de projet de base de données**

**Phase 2 – Enjoy ! Restauration à domicile :**

**Noms des membres du binôme :**

- PACKIYANATHAN Abinayan
- MITH Danny

**1) Présentation**

Notre projet consistait à réaliser un site web fonctionnel inspiré d'applications comme UberEats. L'objectif était de mettre en relation des restaurants, des livreurs et des clients autour d'un système de commande.

Nous avons voulu faire un site simple et efficace : l'interface est épurée avec peu de boutons pour ne pas perdre l'utilisateur. On peut s'y connecter en tant que client ou livreur et utiliser les fonctionnalite principal commandez,mettre des avis , et chercher des restaurants grâce à une barre de recherche et des filtres pour le client et les livreurs peuvent visualiser l'ensemble des commandes en attente dans les villes qu'ils couvrent, et choisir librement celles qu'ils souhaitent prendre en charge.

**2) Justification des choix de modélisation et le modèle relationnel**

- La gestion des Plats est centralisée, l'association Menu (Propose_CARTE) les liant aux Restaurants. Le cœur du système est la Commande, passée par un Client à un Restaurant et assignée à un Livreur. L'association Commande_Plat (Contient) détaille chaque commande et capture le prix au moment de l'achat.
- De plus la livraison se fera par un seul livreur dans le schéma relationnel et le SQL afin de simplifier la table livraison qui prendra seulement un id_livreur. Et l'entité information se fait absorber par les cardinaux de client et livreur, donc ils prennent les « informations de la table INFORMATION » dans leurs tables SQL respectifs de plus cela simplifie la gestion de nos données.
- On a mis id_adresse dans la table client qui est lui-même avec id_ville sur SQL afin de rendre lisible et simple d'accès le lieu de résidence ce qui est un peu différent de notre schéma entité-association.
- Enfin, les Avis (uniques par commande) et les MotsCles (via Resto_MotsCles) enrichissent les données des restaurants, tandis que Livreur_Ville (Travaille) gère la logistique des livraisons.

---

**Ville** (**PK** id_ville, nom_ville, code_postal)

**Adresse** (**PK** id_adresse, num_rue, nom_rue, **FK** id_ville)

-> **FK** Adresse(id_ville) REFERENCES Ville(id_ville)

**Client** (**PK** id_client, email, mot_de_passe, nom, prenom, telephone, carte_bancaire, points_fidelite, **FK** id_adresse, **FK** id_parrain)

-> **FK** Client(id_adresse) REFERENCES Adresse(id_adresse)

-> **FK** Client(id_parrain) REFERENCES Client(id_client)

**Livreur** (**PK** id_livreur, matricule, mot_de_passe, nom, prenom, telephone, etat_service)

**Livreur_Ville**(**PK FK** id_livreur, **PK FK** id_ville)

-> **FK** Livreur_Ville(id_livreur) REFERENCES Livreur(id_livreur)

-> **FK** Livreur_Ville(id_ville) REFERENCES Ville(id_ville)

**Restaurant** (**PK** id_resto, nom, horaires, frais_livraison, statut_exceptionnel, **FK** id_adresse)

-> **FK** Restaurant(id_adresse) REFERENCES Adresse(id_adresse)

**MotsCles** (**PK** libelle_mot_cle)

**Resto_MotsCles**(**PK FK** id_resto, **PK FK** libelle_mot_cle)

-> **FK** Resto_MotsCles(id_resto) REFERENCES Restaurant(id_resto)

-> **FK** Resto_MotsCles(libelle_mot_cle) REFERENCES MotsCles(libelle_mot_cle)

**Plat** (**PK** id_plat, nom, prix, description, photo_url)

**Menu** (**PK FK** id_resto, **PK FK** id_plat)

-> **FK** Menu(id_resto) REFERENCES Restaurant(id_resto)

-> **FK** Menu(id_plat) REFERENCES Plat(id_plat)

**Commande** (**PK** id_commande, date_heure_commande, date_heure_livraison, statut_commande, montant_total, **FK** id_client, **FK** id_livreur, **FK** id_resto)

-> **FK** Commande(id_client) REFERENCES Client(id_client)

-> **FK** Commande(id_livreur) REFERENCES Livreur(id_livreur)

-> **FK** Commande(id_resto) REFERENCES Restaurant(id_resto)

**Commande_Plat** (**PK FK** id_commande, **PK FK** id_plat, quantite, prix_unitaire_capture)

-> **FK** Commande_Plat(id_commande) REFERENCES Commande(id_commande)

-> **FK** Commande_Plat(id_plat) REFERENCES Plat(id_plat)

**Avis** (**PK** id_avis, note, commentaire, **FK** id_client, **FK** id_commande)

-> **FK** Avis(id_client) REFERENCES Client(id_client)

-> **FK** Avis(id_commande) REFERENCES Commande(id_commande)

### 3) Types de données utilisés

Les types suivants ont été privilégiés :

- **Clés primaires (id_client, id_resto) :** INT SERIAL pour l'auto-génération d'identifiants uniques.

- **Chaînes de caractères (nom_ville, email) :** VARCHAR(n) (ex : 100, 255) pour la flexibilité des noms, descriptions et emails.
- **Codes et identifiants fixes (code_postal, matricule) :** VARCHAR(n) (ex : 10, 50) pour préserver les zéros initiaux et autoriser les caractères non numériques.
- **Dates et heures (date_heure_commande) :** DATETIME (ou TIMESTAMP) pour un suivi précis du cycle de vie des commandes.
- **Valeurs monétaires (prix, montant_total) :** DECIMAL (p, s) (ex : DECIMAL (8, 2)) pour garantir la précision des calculs monétaires.
- **Statuts et catégories (etat_service, statut_commande) :** ENUM(...) pour restreindre les valeurs à un ensemble de choix prédéfinis, offrant une lisibilité.
- **Booléens (statut_exceptionnel) :** BOOLEAN pour les indicateurs simples (ex : TRUE/FALSE).

**4) Contraintes**

**A) Contraintes mises en place dans la base de données**

- **Clés primaires et unicité :** PRIMARY KEY sur tous les id_*. UNIQUE sur Client.email, Livreur.matricule et Avis.id_commande (un avis par commande).
- **Contraintes d'intégrité référentielle :**
  - FOREIGN KEY : Lient systématiquement les tables (ex : Commande.id_client référence Client.id_client).
  - ON DELETE :
    - RESTRICT (défaut) : Empêche la suppression d'entités (Ville, Client) si elles sont encore référencées.
    - CASCADE : Utilisé sur les tables associatives (ex : Livreur_Ville, Menu) pour nettoyer les données liées.
    - SET NULL : Utilisé sur Commande.id_livreur et Client.id_parrain pour préserver l'entité principale même si la référence est supprimée.
- **Contraintes de valeur :**
  - NOT NULL : Appliqué aux champs essentiels (ex : Ville.nom_ville, Commande.montant_total).
  - DEFAULT : Utilisé pour initialiser des valeurs (ex : Client.points_fidelite à 0).
  - CHECK : CHECK (note >= 0 AND note <= 5) sur Avis.note pour garantir l'intervalle de valeurs.

**B) Contraintes devant être vérifiées au niveau applicatif**

Certaines règles métier complexes doivent être gérées par l'application (Python/Flask) :

- **Logique d'affichage des restaurants :** L'application doit vérifier l'ouverture du restaurant (horaires, statut) ET la disponibilité d'un livreur couvrant les villes du restaurant et du client.
- **Attribution des points de fidélité :** Le parrain reçoit les points (basés sur le montant) lors de la livraison de la commande de son filleul.
- **Prise en charge des commandes :** L'application doit s'assurer qu'un livreur est 'en_service_attente' et qu'il couvre les villes requises avant de prendre une commande.
- **Cohérence Montant Total :** Le montant_total de la commande doit être calculé par l'application (somme des plats + frais_livraison).

**5) Réponse aux demandes spécifiques (Vues)**

Pour des besoins d'analyse et de reporting, quatre vues ont été créées :

1. **VUES_RELEVE_HEBDO_STATS :** Calcule le nombre total et le prix moyen des commandes des 7 derniers jours pour un suivi général.
2. **VUES_RELEVE_HEBDO_VILLES :** Identifie le nombre de commandes par ville de client (sur 7 jours) pour déterminer les zones géographiques les plus actives.
3. **VUES_STATS_SPECIALITES :** Compte les commandes par spécialité (mot-clé) pour identifier les types de cuisine populaires.
4. **VUES_VERIFICATION_AVIS :** Fournit des données (commande, client, restaurant) à des agences externes, en n'exposant que les informations nécessaires pour vérification.

**5)Description technique**

Pour la barre de recherche on a utilisé une requête qui fait une concaténation de plusieurs petites

Condition la structure de la requête de base est :

```
SELECT R.id_resto, R.nom, R.horaires, R.frais_livraison, R.photo_url,
COALESCE(ROUND(AVG(A.note), 1), 0) AS note_moyenne
FROM Restaurant R
LEFT JOIN Resto_MotsCles RM ON R.id_resto = RM.id_resto
LEFT JOIN Commande C ON R.id_resto = C.id_resto
LEFT JOIN Avis A ON C.id_commande = A.id_commande
WHERE 1=1
```

On utilise COALESCE pour ne pas avoir de note NULL, mais 0 par défaut si on n’a pas de note.

Les petites concaténations sont :

```
AND R.nom ILIKE %s
AND RM.libelle_mot_cle = %s
HAVING COALESCE(ROUND(AVG(A.note), 1), 0) >= %s
```

On a mis WHERE 1=1 afin de pouvoir mettre d'autre condition en concaténation par la suite.

## Panier

Pour le système de panier, nous avons utilisé une structure de dictionnaire stockée dans la session Flask avec comme clés les id_plat (en string) :

session["panier"] = { "1": {"id_plat": 1, "quantite": 2, "prix": 12.50, ...}, "2": {"id_plat": 2, "quantite": 1, "prix": 8.00, ...} }

Cette structure permet une recherche rapide pour vérifier si un plat existe déjà dans le panier et facilite l'ajout/modification des quantités.

Pour la gestion du changement de restaurant pour le panier, nous vérifions l'id_resto du premier plat dans le panier :

```
if panier_dict and len(panier_dict) > 0:
    for plat_panier in panier_dict.values():
        if plat_panier["id_resto"] != id_resto_int:
        # Sauvegarder l'ancien panier, vider pour le nouveau restaurant
```

Si l'utilisateur n'ajoute finalement aucun article du nouveau restaurant, nous restaurons automatiquement son ancien panier.

## Inscription

Pour le système d'inscription, nous gérons la cascade de création des entités :

1. Vérifier/créer la Ville
2. Vérifier/créer l'Adresse
3. Vérifier le parrain (optionnel)
4. Créer le Client avec toutes les infos

Cette approche évite les doublons dans les tables Ville et Adresse.

## Annulation

L'annulation de commande vérifie que : 1. L'utilisateur est connecté 2. La commande appartient bien au client 3. Le statut est 'en_attente' (pas encore en préparation) 4. Retire les points de fidélité du client et de son parrain si applicable

## Connexion livreur

Pour l'authentification des livreurs, nous utilisons bcrypt pour hasher les mots(pour client aussi) de passe et stockons dans la session : matricule, nom_liv, et id_livreur.

Le statut du livreur (hors_service, en_service_attente, en_service_course) détermine quelles commandes il peut voir : - en_service_attente : voit toutes les commandes en attente dans ses villes - en_service_course : voit uniquement sa commande en cours - hors_service : ne voit aucune commande

Nous avons pris en compte les remarques du chargé de TD, il y a donc un bouton déconnexion pour le livreur afin de ne pas pouvoir accéder à la page en modifiant l'URL, avoir la barre de recherche qui fonctionne correctement avec les mots clés associés aux restaurants et nous avons rempli la BDD.

**6)Piste d'améliorations**

- Pouvoir modifier sa zone de travail en tant que livreur
- Pouvoir modifier sa ville en tant que client
- Ajouter un système de promotions avec les points de fidélité

**7)Organisation**

- On est sur du 50/50 :
  Danny a plus touché les fonctions accueil, recherche, liste de restaurant, page par restaurant, profil, confirmer livraison et avis

  Abinayan s'est concentré sur les fonctions suivantes : Inscription, verif, connexion, connexion privée, annuler la commande, page pro, prendre commande

8) Difficultés rencontrées

Lors du développement, nous avons rencontré plusieurs défis techniques :

Gestion du panier et changement de restaurant : Le principal défi était de gérer le cas où un utilisateur ajoute des plats d'un restaurant, puis visite un autre restaurant. Nous avons dû implémenter une logique de sauvegarde temporaire de l'ancien panier au cas où l'utilisateur ne finalise pas sa nouvelle sélection.